

Lab Manual: ETL with PySpark - GCS to BigQuery using Databricks

Objective

By the end of this lab, you will:

- Connect Databricks to Google Cloud Storage (GCS).
- Extract data from GCS into a PySpark DataFrame.
- Transform the data using PySpark.
- Load the transformed results into BigQuery.

This is a common **cloud-native ETL pipeline** pattern.

Prerequisites

Before starting, ensure you have the following:

1. **Databricks Cluster** with:
 - a. GCS connector configured (via **service account key** or **Databricks Secrets**).
 - b. BigQuery Spark Connector installed.

To install the BigQuery connector:

- c. Go to **Compute** → **your cluster** → **Libraries** → **Install New** → **Maven**
- d. Add this coordinate:

`com.google.cloud.spark:spark-bigquery-with-dependencies_2.12:0.29.0`

2. **Google Cloud Setup:**
 - a. A **GCS bucket** containing your sample CSV data (e.g., `gs://my-gcs-bucket/sample_data.csv`).
 - b. A **BigQuery dataset** ready (e.g., `my_dataset`).

Step 1: Initialise Spark Session

```
from pyspark.sql import SparkSession

# Create or retrieve the Spark session
spark =
SparkSession.builder.appName("PySpark_GCS_to_BQ").getOrCreate()
```

This sets up a Spark session named PySpark_GCS_to_BQ.

Step 2: Extract Data from GCS

```
# Define GCS path
gcs_path = "gs://my-gcs-bucket/sample_data.csv"

# Read CSV file from GCS into a DataFrame
df = spark.read.csv(gcs_path, header=True, inferSchema=True)
```

- `header=True` → first row is treated as column names.
- `inferSchema=True` → Spark automatically detects column data types.

Step 3: Transform Data

```
# Filter rows and rename a column
transformed_df = df.filter(df.amount > 200).withColumnRenamed("id",
"customer_id")
```

- Filters rows where `amount > 200`.
- Renames `id` column to `customer_id`.

Step 4: Load Data into BigQuery

```
# Write transformed data to BigQuery
transformed_df.write \
    .format("bigquery") \
```

```
.option("table", "my-gcp-project.my_dataset.filtered_data") \
.mode("overwrite") \
.save()
```

- Uses the **BigQuery connector**.
- Saves data into table `my_dataset.filtered_data` in your GCP project.
- `mode("overwrite")` replaces any existing table data.

Step 5: Preview Transformed Data

```
# Show first 20 rows in Databricks
transformed_df.show()
```

Quickly check results before loading into BigQuery.

Step 6: Run and Verify

Attach Notebook to Cluster

- Ensure your notebook is attached to an active cluster.
- Confirm that:
 - GCS access is configured (service account or Databricks secret).
 - BigQuery connector is installed.

Test GCS Connectivity

```
# Test read from GCS
spark.read.csv("gs://my-gcs-bucket/sample_data.csv",
header=True).show(5)
```

- If you see sample rows → GCS access works.
- If you get an authentication error → check your service account key or Databricks secret.

Run the Full Notebook

1. **Extract:** Read CSV from GCS into PySpark DataFrame.
2. **Transform:** Filter rows & rename column.

3. **Load:** Write results to BigQuery.
4. **Preview:** Confirm with `.show()`.

Step 7: Verify in BigQuery Console

1. Open the **BigQuery Console** in your GCP project.
2. Navigate to your dataset (`my_dataset`) and table (`filtered_data`).
3. Run the query:

```
SELECT COUNT(*)  
FROM `my-gcp-project.my_dataset.filtered_data`;
```
4. Verify that the row count matches the filtered data you saw in Databricks.

Verification Checklist

- Cluster has GCS + BigQuery connectors.
- Data read from GCS successfully.
- Transformation applied (`amount > 200`, column renamed).
- Data written to BigQuery table.
- Results match in both Databricks and BigQuery.

Summary

In this lab, you:

- Connected Databricks to **Google Cloud Storage**.
- Extracted CSV data into PySpark.
- Transformed data (filter + rename).
- Loaded results into **BigQuery**.
- Verified transformations in both Databricks and BigQuery.

This pattern — **GCS → PySpark (Databricks) → BigQuery** — is a real-world cloud-native ETL workflow. With minor connector changes, you can adapt the same pipeline for:

- **AWS S3 → Redshift**
- **Azure Blob → Synapse**